

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №6
Реализация принципов дизайна и организации компонентов
тема

Преподаватель

подпись, дата

Д. В. Грузенкин
инициалы, фамилия

Студент КИ23-17/1Б, 0323286822

номер группы, зачетной книжки

подпись, дата

В. В. Белоглазов
инициалы, фамилия

Красноярск 2026

1 Цель

Провести сбор и анализ требований к системе, архитектура которой будет разрабатываться в этом семестре.

2 Задачи

Составить список требований для проектируемой системы на следующих уровнях:

- бизнес-требования (для их составления вам понадобится пусть и вымышленный, но конкретный заказчик);
- требования пользователей;
- функциональные требования;
- нефункциональные требования.

Описать источники и методы получения требований, которые бы вы использовали в реальной работе с заказчиком по проектируемой системе.

3 Ход выполнения

3.1 Выбор и описание системы для проектирования

В качестве системы для проектирования был выбран проект «Astroll» — веб-ориентированная система для проведения настольных ролевых игр онлайн (VTT, Virtual Tabletop). Система предназначена для хранения и редактирования персонажей и заметок пользователей и организации игровой сессии в формате “комнаты”, где участники видят общую доску, совершают игровые действия и синхронно взаимодействуют в реальном времени.

Назначение системы Astroll заключается в предоставлении мастеру и игрокам удобного пространства для ведения игровой сессии: общей доски, добавления и редактирования изображений, создание рисунков на доске, добавление и редактирование изображений, управления персонажами(совершение бросков, перемещение по доске) и отображения их токенов(иконок), выполнения бросков кубов, а также хранения

пользовательских данных (профиль, друзья, персонажи, заметки) и сохранения/загрузки досок, создание и редактирование персонажей.

В системе предусмотрены две основные роли: Мастер и Игрок.

Мастер (создатель комнаты) организует сессию, ведет заметки, сохраняет и загружает доски, а также управляет участниками комнаты: может ограничивать права (режим “только просмотр”) и исключать игроков из комнаты.

Игрок подключается к комнате по ссылке, просматривает и при наличии прав редактирует доску, использует лист персонажа и участвует в игровых действиях (в том числе бросках кубов), результаты которых отображаются в комнате.

Система включает следующие основные функции:

- регистрация и авторизация пользователя;
- управление интерактивной доской (добавление пользователей, управление правами пользователя, добавление изображений и текста, рисовать на доске, добавлять и редактировать фигуры, сделать бросок кубов за выбранного персонажа, добавить токен-иконку персонажа) ;
- управление профилем (никнейм, аватар), поиск пользователей, добавление/удаление друзей;
- создание, просмотр, редактирование и удаление персонажей;
- подключение к игровой комнате и совместная работа на доске;
- синхронизация состояния доски и событий в реальном времени;
- броски кубов, зависящие от характеристик персонажа и просмотр истории бросков;
- ведение заметок мастера (журнал страниц) и сохранение их вместе со “снапшотом” доски;
- сохранение/загрузка досок (snapshots) из базы данных.

UML-диаграмма вариантов использования представлена на рисунках 1 и 2.

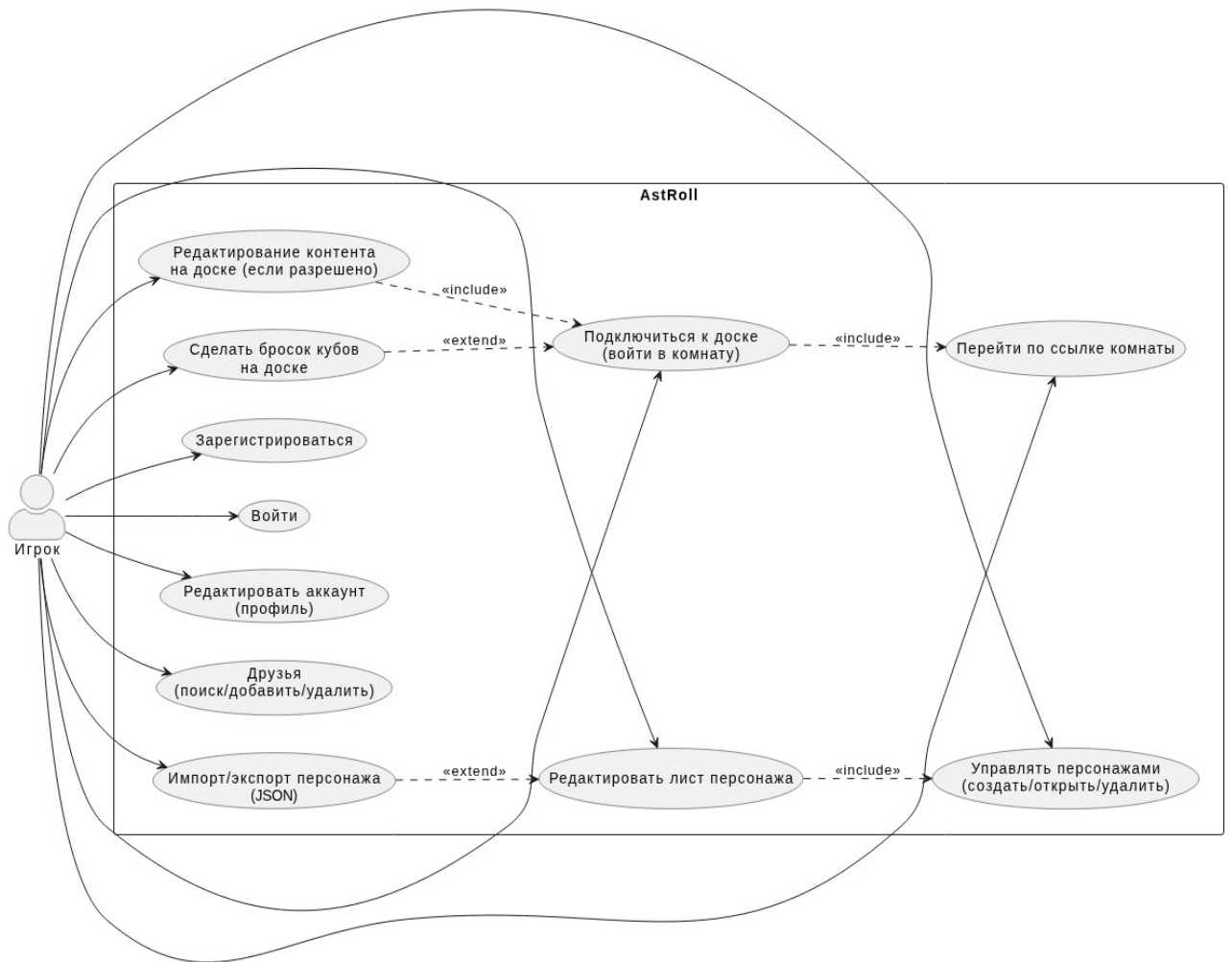


Рисунок 1 – UML-диаграмма вариантов использования системы (только для роли «Игрок»)

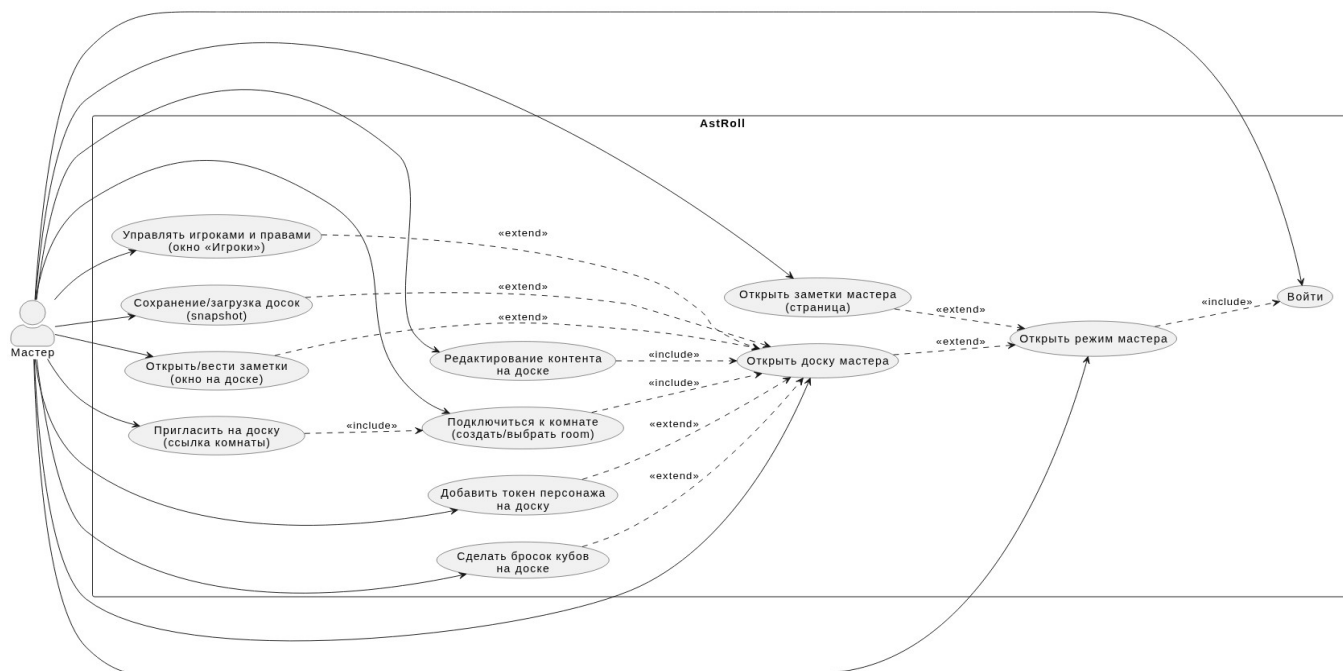


Рисунок 2 – UML-диаграмма вариантов использования системы (только для ролей «Мастер»)

3.2 Составление списка бизнес-требований

В качестве заказчика выступает вымышленная организация — клуб настольных ролевых игр «Энросс-Клуб», которая проводит регулярные сессии и хочет иметь единый веб-инструмент для:

- стандартизированного ведения листов персонажей;
- совместной онлайн-доски и реалтайм-взаимодействия;
- уменьшения ручной рутины мастера и игроков.

Для получения требований в реальной разработке целесообразны:

- интервью со стейкхолдерами (организатор, мастера, игроки);
- наблюдение за проведением сессии (офлайн/онлайн);
- анализ артефактов: шаблоны листов, правила, привычные инструменты (Discord, Google Docs);
- воркшоп по приоритизации (MoSCoW), уточнение MVP.

Проект: «веб-система для проведения настольных ролевых игр»

Дата проведения: 21.02.2026

Интервьюер: Белоглазов Владислав Владимирович

Интервьюируемые:

- Стейкхолдер 1 – Кузнецов Артём Сергеевич, организатор клуба ведущий игр
- Стейкхолдер 2 – Кузнецова Алина Игоревна, соорганизатор клуба, активный игрок и ведущий игр

Проблематика: Отсутствие централизованного инструмента для проведения игр и хранения листов персонажей, сложность ведения онлайн-сессий (карты/схемы/броски)

Вопрос № 1:

Тайминг вопроса: 00:02-00:07

Текст вопроса: Как сейчас ведутся листы персонажей?

Таймкод: 00:03

Ответы на вопрос:

Стейкхолдер 1 – Сейчас хранение осуществляется в файлах/таблицах (Google Docs/Sheets) или в физическом формате на бумаге, у каждого игрока отдельная копия, мастеру сложно контролировать и отслеживать персонажей игроков, потому что у всех игроков лист персонажа в разных форматах.

Стейкхолдер 2 – Меня не устраивает хранить данные персонажа в Google Docs, я хочу чтобы лист персонажа работал на погружение во время игры, а сухая таблица с цифрами мне нравится меньше, чем бумажный вариант с рукописными записями.

Вопрос № 2:

Тайминг вопроса: 00:07-00:13

Текст вопроса: Что наиболее мешает во время игровой сессии?

Таймкод: 00:08

Ответы на вопрос:

Стейкхолдер 1 – Во-первых, это время. Процесс определения характеристик для бросков и контроль игроков занимает огромное количество

времени. Во-вторых, человеческий фактор – кто-то непременно перепутает куда смотреть для броска или ошибется в расчетах. Это приходится перепроверять и отвлекаться от повествования.

Стейкхолдер 2 – Пересылка картинок для описания мастером, это занимает слишком много времени и приходится отвлекаться на чат мессенджера.

Вопрос № 3

Тайминг вопроса: 00:13-00:18

Текст вопроса: Какие функции критичны для MVP?

Таймкод: 00:14

Ответы на вопрос:

Стейкхолдер 1 – Удобная в использовании интерактивная доска с отслеживанием бросков игроков.

Стейкхолдер 2 – Удобный лист персонажа с красивым визуалом и автосохранением.

Вопрос № 4

Тайминг вопроса: 00:18-00:24

Текст вопроса: Как измерить успех внедрения?

Таймкод: 00:19

Ответы на вопрос:

Стейкхолдер 1 – Сокращение времени на подготовку к игре и механические процессы(на 25-30%), такие как броски кубов или редактирование листа персонажа. Также сократить время синхронизации игрового поля у игроков. Необходимо чтобы доска работала синхронно у всех игроков, с задежкой в пределах нормы(2-3 с).

По результатам проведенного интервью был сформулирован список бизнес-требований, каждый пункт списка содержит цели предприятия и ожидаемый эффект от возможного решения.

Бизнес-требования заказчика:

1. Стандартизировать ведение листов— система должна предоставить единый формат листа персонажа, чтобы информация была полной и согласованной.
2. Сократить время на подготовку и ведение сессии — уменьшить ручную рутину и поиск данных по персонажам.
3. Обеспечить онлайн-взаимодействие— предоставить доску (VTT) и синхронизацию действий в реальном времени.
4. Обеспечить сохранность данных — листы и доски должны храниться в базе и быть доступны после перезапуска/перезахода.
5. Сделать продукт доступным и простым — работа в браузере, понятная навигация, минимум шагов для старта.

3.3 Выявление требований пользователя

Для выявления требований пользователей была проведена вымышленная беседа с посетителями клуба настольных игр, среди которых как мастера так и игроки.

Интервьюер: Белоглазов Владислав Владимирович

- Пользователь 1:

Вопрос № 1: Представьтесь, пожалуйста. Кто вы и чем занимаетесь?

Ответ на вопрос: Я игрок. Обычно веду одного-двух персонажей в разных кампаниях, между сессиями обновляю лист, на игре быстро смотрю значения характеристик и заметки. Я хочу иметь возможность редактировать свой профиль, а также искать и добавлять других пользователей в друзья.

Вопрос № 2: Что вам было бы удобно иметь в новой системе?

Ответ на вопрос: Хочу, чтобы лист персонажа был в одном месте и всегда актуальный. Чтобы можно было быстро править значения прямо во время игры и не думать, что “у меня старая копия”. Ещё важно уметь сохранить лист в файл и перенести/отправить мастеру при необходимости.

Вопрос № 3: Нужна ли вам совместная доска и участие в комнате мастера?

Ответ на вопрос: В онлайн хочется, чтобы у всех была одна и та же доска, без пересылки скриншотов. Мне удобно просто зайти в комнату и видеть изменения в реальном времени.

- Пользователь 2:

Вопрос № 1: Представьтесь, пожалуйста. Кто вы и чем занимаетесь?

Ответ на вопрос: Я мастер. Веду сессии, готовлю материалы, управляю сценами/боем, слежу за тем, чтобы у всех была одинаковая информация и чтобы игра не тормозилась из-за технических мелочей.

Вопрос № 2: Что для вас важно в новой системе?

Ответ на вопрос: Мне нужна доска, на которой я могу вести сцену и которую видят игроки. Важно, чтобы доска не “разъезжалась” у разных людей, чтобы можно было быстро продолжить с того же состояния. Также хочу видеть, кто подключён к комнате, и иметь возможность отключить интерактивность, если кто-то мешает (только просмотр), или кикнуть..

- Пользователь 3:

Вопрос № 1: Представьтесь, пожалуйста. Кто вы и чем занимаетесь в клубе?

Ответ на вопрос: Я игрок. Я редко играю в настольные игры и обычно мне требуется большое количество времени чтобы разобраться в правилах и листе персонажа.

Вопрос № 2: Что для вас важно в новой системе?

Ответ на вопрос: Мне было бы удобно с понятным листом персонажа и кнопкой броска кубов, которая сама считает сколько кубов нужно кинуть и какой результат.

Полученное интервью-опрос можно использовать для документирования требований пользователей. Для их представления было принято решение использовать пользовательские истории (User Story) для двух ролей:

1) Мастер – главный пользователь системы, чья задача проводить игры в интерактивной доске, собирать вместе игроков, помогать им генерировать персонажей.

- КАК мастер, Я ХОЧУ удобный редактируемый лист персонажа, ЧТОБЫ было удобнее создавать персонажей для игроков;

- КАК мастер, Я ХОЧУ иметь интерактивную доску с масштабированием/перемещением, ЧТОБЫ вести сцены и бои;

- КАК мастер, Я ХОЧУ править доступом к доске (readonly) и исключать нарушителей (kick), ЧТОБЫ контролировать сессию;

- КАК мастер, Я ХОЧУ видеть историю бросков, ЧТОБЫ разбирать спорные моменты.

2) Игрок – человек, который будет хранить своих персонажей и участвовать в играх, которые проводит мастер;

- КАК игрок, Я ХОЧУ зарегистрироваться и войти в систему, ЧТОБЫ всегда иметь доступ к своим данным и персонажам;

- КАК игрок, Я ХОЧУ создавать несколько персонажей, ЧТОБЫ участвовать в разных кампаниях;

- КАК игрок, Я ХОЧУ редактировать лист персонажа, ЧТОБЫ быстро обновлять информацию во время игры;

- КАК игрок, Я ХОЧУ подключаться к комнате и видеть доску в реальном времени, ЧТОБЫ играть онлайн.

3.4 Составление списка функциональных требований

Функциональные требования были получены декомпозицией бизнес-требований и требований пользователей. Список функциональных требований:

FRQ-1. Аутентификация и авторизация

- FRQ-1.1 Система должна предоставлять пользователю возможность регистрации по login и паролю;

- FRQ-1.2 При регистрации система должна валидировать login и минимальную длину пароля (6 символов);

- FRQ-1.3 Система должна предотвращать повторную регистрацию на уже существующий login и возвращать понятную ошибку;

- FRQ-1.4 Система должна предоставлять пользователю возможность входа по login и паролю;
- FRQ-1.5 При входе система должна возвращать токен авторизации (JWT) и данные пользователя;
- FRQ-1.6 Система должна использовать JWT-токен в заголовке `Authorization: Bearer <token>` для авторизации запросов к защищённым ресурсам;
- FRQ-1.7 Система должна ограничивать доступ к основным разделам UI без авторизации (перенаправление на страницу входа);
- FRQ-1.8 Система должна позволять пользователю завершить сессию (logout) путём удаления локального токена на клиенте.

FRQ-2. Профиль пользователя и друзья

- FRQ-2.1 Система должна предоставлять просмотр собственного профиля;
- FRQ-2.2 Система должна позволять изменять nickname и avatar;
- FRQ-2.3 Система должна позволять изменять login и пароль;
- FRQ-2.4 При смене пароля система должна сохранять только хеш (без хранения исходного пароля);
- FRQ-2.5 Система должна предоставлять поиск пользователей по строке запроса (по login/нику);
- FRQ-2.6 Система должна позволять добавлять пользователя в друзья и удалять из друзей;
- FRQ-2.7 Система должна запрещать добавлять в друзья самого себя;
- FRQ-2.8 Система должна позволять просматривать профиль другого пользователя по id (в рамках авторизованного доступа).

FRQ-3. Персонажи

- FRQ-3.1 Система должна предоставлять список персонажей текущего пользователя;

- FRQ-3.2 Система должна возвращать список персонажей в отсортированном виде (например, по времени последнего изменения);
- FRQ-3.3 Система должна позволять создать нового персонажа;
- FRQ-3.4 При создании персонажа система должна инициализировать базовые данные листа (дефолтный `sheetData`);
- FRQ-3.5 Система должна позволять получить персонажа по id;
- FRQ-3.6 Система должна запрещать доступ к персонажу по id, если он принадлежит другому пользователю;
- FRQ-3.7 Система должна позволять обновлять персонажа частично (PATCH): имя, `sheetData`, `schemaVersion`;
- FRQ-3.8 Система должна удалять персонажа и связанные данные по запросу владельца (DELETE).

FRQ-4. Лист персонажа

- FRQ-4.1 Система должна хранить лист персонажа в JSON-формате `sheetData`;
- FRQ-4.2 Формат `sheetData` должен включать метаданные (`meta`) с `schemaVersion` и датой создания;
- FRQ-4.3 Система должна поддерживать многостраничность листа (минимум page1 + дополнительные страницы);
- FRQ-4.4 Система должна хранить портреты персонажа (dataUrl) и поддерживать кадрирование/выбор основного портрета;
- FRQ-4.5 Система должна поддерживать авто-сохранение (debounce) и отправку изменений в API;
- FRQ-4.6 Система должна показывать пользователю статус сохранения (например, “Сохранение...”);
- FRQ-4.7 Система должна поддерживать импорт листа из JSON файла (с проверкой структуры данных);
- FRQ-4.8 Система должна поддерживать экспорт листа в JSON файл;

- FRQ-4.9 Система должна обеспечивать совместимость данных при изменении структуры (merge, schemaVersion).

FRQ-5. Режим мастера (комнаты, доска, реалтайм)

- FRQ-5.1 Система должна позволять подключаться к комнате по `roomId`;
- FRQ-5.2 Система должна поддерживать присоединение к комнате как с токеном (авторизованный пользователь);
- FRQ-5.3 Система должна синхронизировать состояние доски между участниками комнаты в реальном времени (Socket.IO);
- FRQ-5.4 Система должна предоставлять снимок доски для вновь подключившихся участников (board:get → board:snapshot);
- FRQ-5.5 Система должна хранить (кешировать) последнее состояние доски в рамках комнаты на сервере на время работы процесса;
- FRQ-5.6 Система должна предоставлять список присутствующих игроков (presence/roster) в комнате и обновлять его при входе/выходе;
- FRQ-5.7 Система должна определять “мастера/хоста” комнаты и давать ему права управления;
- FRQ-5.8 Система должна позволять мастреу комнаты переключать игрокам readonly и выполнять kick;
- FRQ-5.9 Система должна уведомлять участников о смене прав (readonly) и факте исключения (kick).

FRQ-6. Броски и история

- FRQ-6.1 Система должна поддерживать передачу результатов бросков из листа персонажа в комнату (доску);
- FRQ-6.2 Сообщение о броске должно содержать минимум: автора (или “аноним”), тип/описание броска и итоговое значение;
- FRQ-6.3 Система должна отображать историю бросков в UI;
- FRQ-6.4 История бросков должна обновляться в реальном времени без перезагрузки страницы.

FRQ-7. Хранение досок в БД

- FRQ-7.1 Система должна сохранять доски пользователя как сущность `Board` с JSON-снимком `snapshot`;
- FRQ-7.2 Система должна предоставлять CRUD-операции для досок (создать/список/получить/обновить/удалить);
- FRQ-7.3 Система должна запрещать доступ к доскам другого пользователя;
- FRQ-7.4 Система должна позволять переименовывать доску без потери снимка.

FRQ-8. Диагностика

- FRQ-8.1 Система должна предоставлять эндпоинт `GET /api/health` для проверки доступности API и БД;
- FRQ-8.2 Эндпоинт должен возвращать признак доступности базы и понятное сообщение при проблемах;
- FRQ-8.3 Ошибки API должны возвращаться в JSON-формате с человекочитаемым описанием.

3.5 Составление списка нефункциональных требований

NFRQ-1. Безопасность:

- NFRQ-1.1 Система должна хранить пароли в виде безопасного хеша (bcrypt);
- NFRQ-1.2 Система не должна хранить пароли в открытом виде и не должна логировать их;
- NFRQ-1.3 Система должна предотвращать доступ к чужим персонажам и доскам (проверка владельца на сервере);
- NFRQ-1.4 Система должна обеспечивать защищённую передачу данных в продакшене (HTTPS/TLS на уровне хостинга/прокси);

- NFRQ-1.5 Система должна валидировать входные данные на бэкенде (например, Zod) и возвращать 4xx при ошибках валидации;
- NFRQ-1.6 Система должна использовать секрет подписи JWT, настраиваемый через переменные окружения.

NFRQ-2. Удобство использования

- NFRQ-2.1 Интерфейс должен быть веб-ориентированным и доступным в актуальных версиях браузеров: yandex, firefox, opera, google chrom;
- NFRQ-2.2 Основные сценарии (регистрация → создание персонажа → редактирование → режим мастера) должны выполняться без длительного обучения;
- NFRQ-2.3 Авто-сохранение не должно приводить к потере пользовательских данных и не должно мешать вводу;
- NFRQ-2.4 UI должен выдавать понятные сообщения об ошибках (например, “нет соединения с сервером”, “нужна авторизация”).

NFRQ-3. Производительность и реалтайм

- NFRQ-3.1 Система должна быть отзывчивой при типовой нагрузке малой группы (2–8 игроков);
- NFRQ-3.2 Реалтайм-синхронизация доски должна работать без заметных задержек для участников(2-3 с);
- NFRQ-3.3 Операции сохранения листа (PATCH) не должны блокировать интерфейс пользователя.

NFRQ-4. Надёжность

- NFRQ-4.1 Данные персонажей и досок должны сохраняться в БД и быть доступны после перезапуска сервера;
- NFRQ-4.2 Система должна корректно обрабатывать временную недоступность БД и возвращать корректные коды ошибок (например, 503);

- NFRQ-4.3 При проблемах с БД система должна возвращать понятные сообщения, достаточные для диагностики (что проверить в окружении);
- NFRQ-4.4 Система должна сохранять целостность данных при частичных обновлениях (PATCH).

3.6 Выбор архитектурного паттерна

Дальше необходимо было выбрать архитектурный паттерн для данной системы. Выбранный паттерн и обоснование выбора были описаны в виде ADR (Architecture Decision Record):

Запись принятого решения №1: Выбор архитектурного паттерна для системы

Дата: 2026-03-05

Контекст:

Выделены главные функциональные ограничения: низкая задержка и масштабирование по комнатам (NFRQ-2.1, NFRQ-2.2); отказоустойчивость подсистем (NFRQ-3.1); наблюдаемость и управляемые релизы (NFRQ-4.2); развитие доменов с явными границами в коде и возможность горизонтального масштабирования без обязательного дробления на множество сервисов.

Решение:

Модульный монолит: один исполняемый backend и единый цикл выпуска, внутри — доменные модули и общая реляционная база данных; бинарные медиа выносятся во внешнее объектное хранилище.

Целевой набор модулей (каждый отвечает за свой контекст, данные — в общей БД со схемами или соглашениями об именовании):

1. Identity/Auth/Users — регистрация/вход, refresh-сессии, отзыв токенов, профиль, ник, аватар.
2. Friends — заявки в друзья, список друзей.
3. Lobby/Rooms — комнаты, членство, приглашения, права в комнате.
4. Board — хранение и версионирование состояния доски, правила доступа.

5. Dice — броски и история бросков в комнате.

6. Characters — персонажи и данные листов .

Все эти модули работают с одной монолитной базой данных.

Последствия:

Преимущества: один деплой и одна БД — проще эксплуатация, бэкапы и миграции; проще согласованность данных в сценариях комнаты; меньше сетевых контрактов для небольшой команды; модули дают обзорность; при росте нагрузки — горизонтальное масштабирование.

Риски: масштабируется весь процесс целиком; сбой или дефект в модуле теоретически влияют на процесс (смягчается слоями, тестами, мониторингом); нужна дисциплина границ модулей.

Для ASTROLL такой баланс предпочтительнее полноценной микросервисной платформы.

Альтернативы:

В качестве альтернативы **расStream Processing** рассматривалась микросервисная архитектура. Такой подход делает систему менее связанной и даёт возможности масштабирования, а также делает систему более отказоустойчивой, но значительно усложняет инфраструктуру и разработку модулей.

Также в качестве альтернативы модульному монолиту веб-приложения рассматривалась слоистая архитектура. Она проще в реализации, но логика работы выделенных главных модулей становится размазанной по всем слоям, что усложняет поддержку кода и реализацию изменений.

3.7 Реализация карты контекстов по методологии DDD

Далее проект «Astroll» был проанализирован с точки зрения методологии DDD.

Главный домен (бизнес-домен) данной системы — проведение онлайн-настольной ролевой сессии: совместная интерактивная доска (VTT),

синхронизация действий участников в реальном времени, управление комнатой и игровыми событиями (в том числе бросками кубов).

Для него были выделены поддомены:

а) Смысловое ядро:

1) Игровая сессия (VTT). Это ключевая бизнес-ценность системы: мастер и игроки работают в одной «комнате» на общей доске, видят присутствие участников, права доступа и результаты игровых действий без рассинхронизации состояния.

б) Вспомогательные поддомены:

1) Персонажи и листы — хранение, редактирование и импорт/экспорт листов персонажа (включая данные стэнда).

2) Хранение досок — сохранение и загрузка снимков досок пользователя в базе данных.

3) Комнаты и присутствие — идентификатор комнаты, roster участников, назначение хоста (мастера), приглашения.

4) Броски кубов — формирование и трансляция записей о бросках в комнату, история бросков.

5) Друзья — социальные связи между пользователями (добавление/удаление, поиск).

в) Поддомены общего назначения:

1) Учётные записи и роли — регистрация, вход, JWT-авторизация, профиль пользователя. Для аутентификации можно опираться на типовые решения (хеширование паролей, выдача токенов).

Исходя из данных поддоменов и в соответствии с выбранной архитектурой (модульный монолит, см. ADR в работе №3) были определены ограниченные контексты и связи между ними, из которых составлена карта контекстов, показанная на рисунке 3.

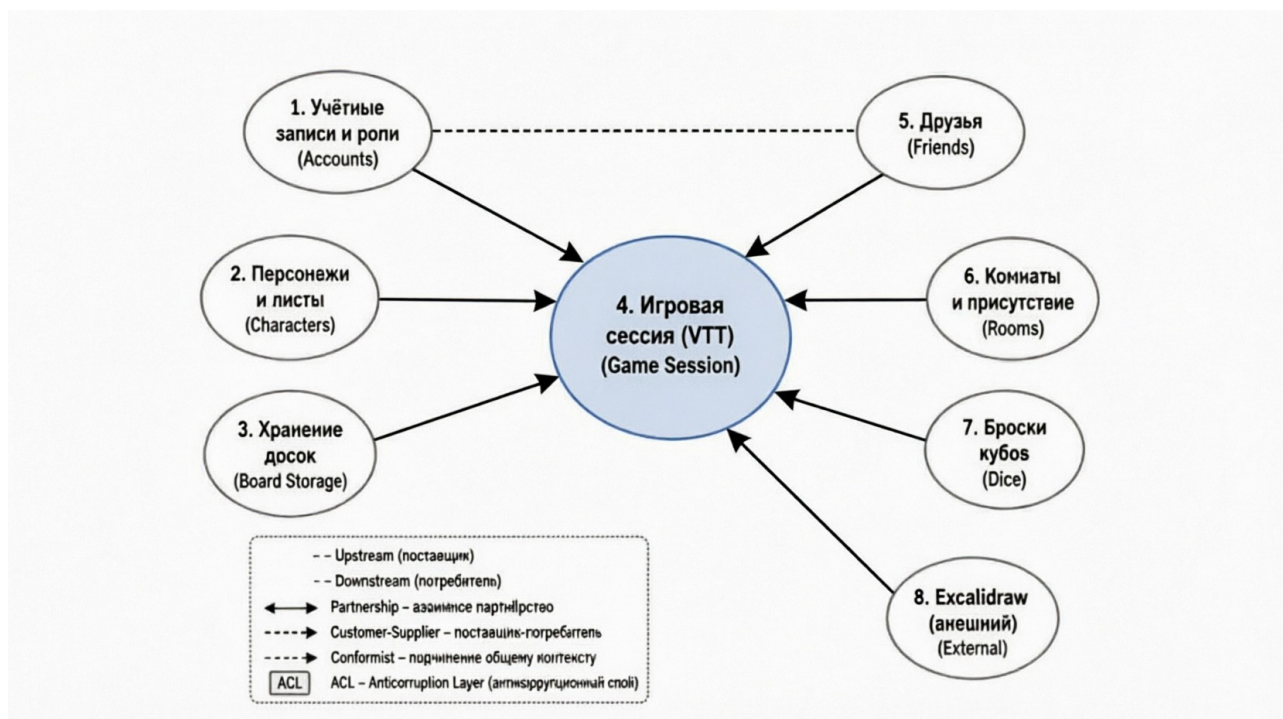


Рисунок 3 – Карта контекста системы по методологии DDD

Ограниченные контексты, приведённые на карте контекстов:

1) Игровая сессия (VTT) — центральный контекст смыслового ядра. Он координирует реалтайм-синхронизацию доски (Socket.IO), применяет права мастера (readonly, kick), использует данные персонажей и снимоты досок. Для остальных контекстов, которым нужны игровые события и состояние сессии, он выступает Upstream.

2) Accounts (учётные записи и роли) — контекст регистрации, входа, профиля и проверки доступа. Он задаёт единую модель пользователя (userId, JWT), которую потребляют Friends, Characters и Game Session, поэтому связи с ними оформлены как Conformist (Downstream подстраивается под модель Accounts).

3) Characters (персонажи и листы) — контекст CRUD персонажей, многостраничного листа, портретов, авто-сохранения и импорта/экспорта JSON. Он поставяет sheetData и связанные данные в Game Session для бросков и токенов на доске — связь Customer-Supplier (Characters — поставщик, Game Session — потребитель).

4) Board Storage (хранение досок) — контекст сохранения снимков Excalidraw в сущности Board. Загруженные снимки передаются в активную сессию — связь Customer-Supplier с Game Session.

5) Rooms (комнаты и присутствие) — контекст roomId, roster, claimHost и приглашений. Game Session опирается на него как на поставщика метаданных комнаты — Customer-Supplier (Rooms — поставщик).

6) Dice (броски кубов) — контекст roll:entry и истории бросков в комнате. С Game Session он связан Partnership: оба тесно участвуют в игровом процессе и развиваются согласованно; при этом формат событий изолируется внутри домена.

7) Friends (друзья) — контекст заявок и списка друзей, поиска пользователей. Подчиняется модели Accounts (Conformist).

8) Excalidraw (внешний) — внешняя библиотека холста для отрисовки и редактирования доски. Для связи с Game Session используется Anti-Corruption Layer: ядро не должно зависеть от деталей API и внутреннего состояния виджета Excalidraw.

3.8 Диаграмма контекста нотации C4

Для смыслового ядра и одного из вспомогательных поддоменов построены диаграммы контекста нотации C4. Первая диаграмма показана на рисунке 4.



Рисунок 4 – Диаграмма контекста нотации C4 для смыслового ядра системы

Центральная система на диаграмме — смысловое ядро «Игровая сессия (VTT)». Её основные пользователи — персоны Мастер и Игрок. Мастер ведёт сессию, управляет комнатой, сохраняет доски и сценой; игрок подключается к комнате, редактирует лист и при наличии прав — доску.

Ядро получает аутентификацию из поддомена общего назначения «Учётные записи» (регистрация, JWT, профиль), данные комнаты и roster — из «Комнаты», sheetData для бросков и токенов — из «Персонажи и листы», снапшоты — из «Хранение досок», события бросков обменивает с «Броски кубов». Для отображения и редактирования поля используется внешняя система Excalidraw через Canvas API и ACL.

На рисунке 5 показана диаграмма контекста нотации C4 для вспомогательного поддомена «Персонажи и листы».



Рисунок 5 – Диаграмма контекста нотации C4 для вспомогательного поддомена системы

Центральная система на диаграмме — вспомогательный поддомен «Персонажи и листы»: CRUD персонажей, многостраничный лист, портреты и стэнды, авто-сохранение, импорт/экспорт JSON. Игрок создаёт и редактирует персонажей между сессиями и правит лист во время игры.

Контекст опирается на «Учётные записи» (владелец персонажа, JWT, проверка доступа по `userId`) и передаёт `sheetData` в смысловое ядро «Игровая сессия» для бросков и токенов на доске. Подсказки по стэндам берутся из внешнего артефакта «Справочник Энросс» (`GuideForStadns.md`) и отображаются в UI.

3.9 Реализация паттернов проектирования

Далее в проектируемой системе Astroll предусматривается внедрение по одному паттерну каждого вида в смысловом ядре приложения. Код целевой реализации следует размещать в слое домена и прикладных сервисов, изолированном от конкретных транспортов (HTTP, WebSocket) и от формата внешних библиотек визуализации доски.

3.9.1 Порождающий паттерн

Для порождающего паттерна в смысловом ядре системы Astroll целесообразно выбрать паттерн «Фабричный метод». Его UML-диаграмма классов должна быть приведена на рисунке 6.

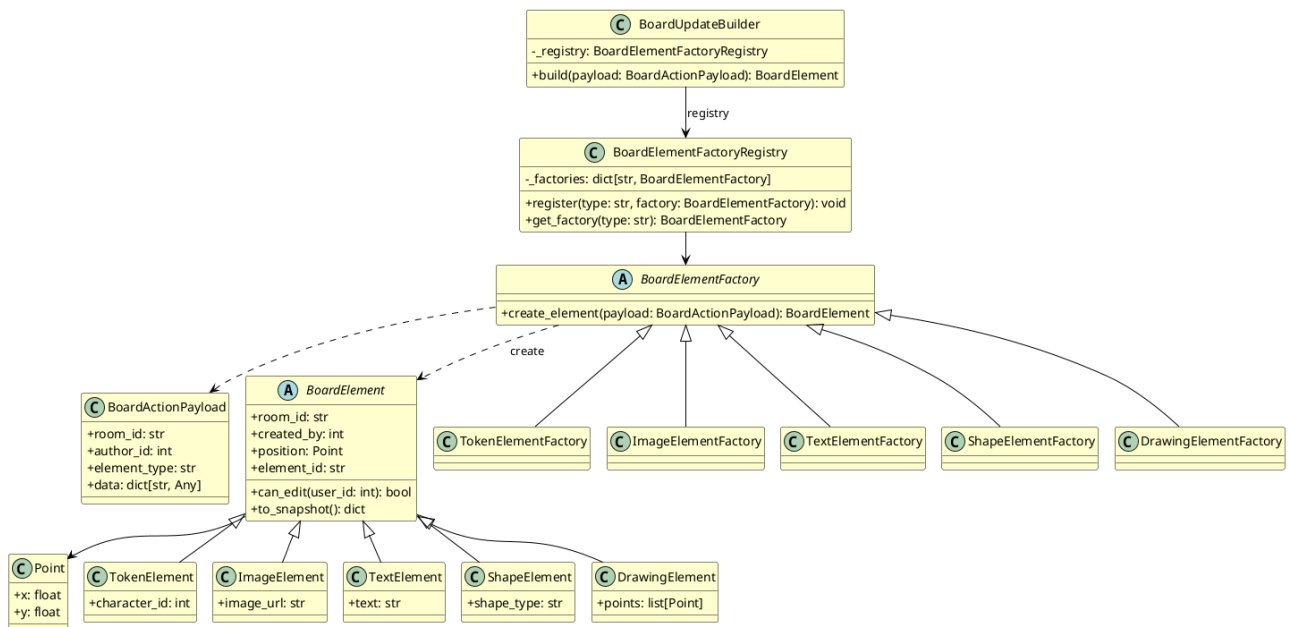


Рисунок 6 – Диаграмма классов реализации паттерна «Фабричный метод»

Паттерн следует применить в контексте игровой сессии и доски: доменные объекты элементов поля (токен персонажа, фоновое изображение, примитив рисования, текстовая метка и т.п.) должны создаваться через общий интерфейс фабрики, а конкретные фабрики — порождать согласованные типы элементов без дублирования логики разбора входных данных. Клиентом выступает сервис сценария «применить действие к доске» или построитель обновления состояния комнаты, который по типу входного намерения (например, добавить токен, вставить изображение) делегирует создание объекта соответствующей конкретной фабрике. Абстрактный продукт описывает общий контракт элемента доски для правил видимости, синхронизации и проверки прав мастера и игрока.

Выбор паттерна обоснован тем, что в требованиях к Astroll перечислены разнородные объекты доски и действия над ними, при этом набор типов элементов со временем расширяется, а клиентский код сценариев не должен зависеть от конкретных классов каждого вида.

Порождающий паттерн «Абстрактная фабрика» в данном месте использовать нецелесообразно, если не требуется одновременно подменять целые семейства взаимосвязанных объектов разных платформ или тем оформления; для поэлементного расширения доски достаточно фабричного метода с одним продуктом на ветвь иерархии.

3.9.2 Структурный паттерн

Для структурного паттерна в смысловом ядре системы Astroll целесообразно выбрать паттерн «Адаптер». Его UML-диаграмма классов должна быть приведена на рисунке 7.

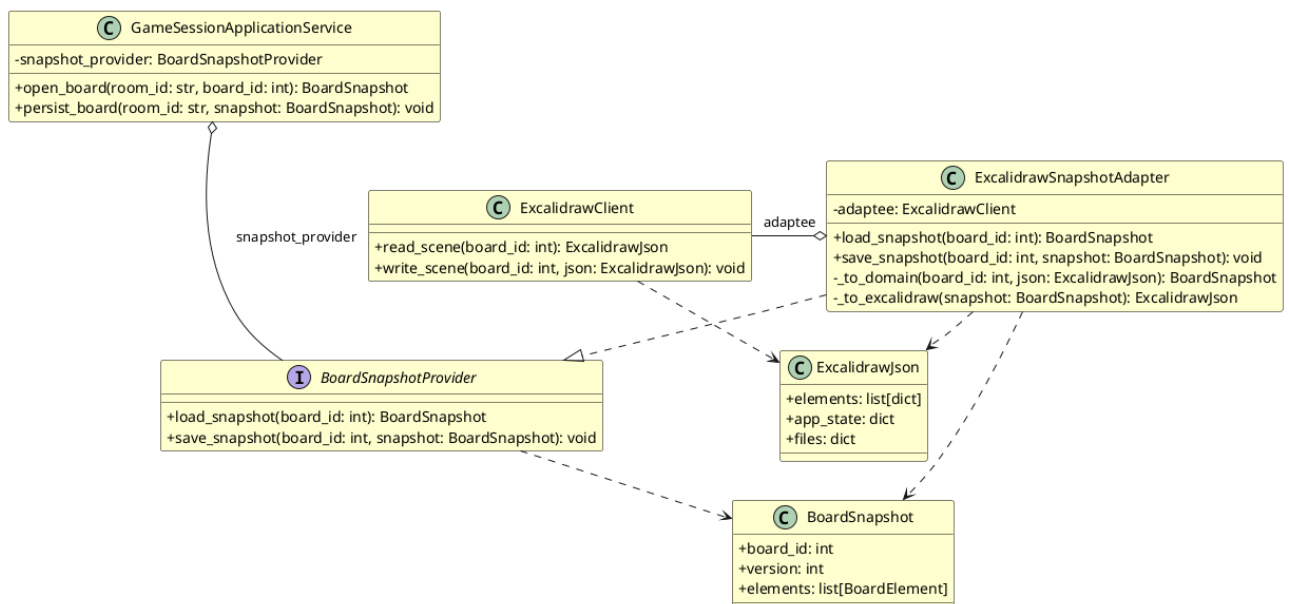


Рисунок 7 – Диаграмма классов реализации паттерна «Адаптер»

На границе ядра и снимота доски: формат сторонней библиотеки (например, Excalidraw) не совпадает с моделью агрегата «комната и доска». Порт внутреннего уровня и адаптер переводят JSON снимота в доменные структуры и обратно при сохранении и загрузке (требования ПрПО5). Сообщения реального времени не должны попадать в домен сырыми словарями — их приводят к командам или DTO.

Выбор обоснован изоляцией правил комнаты и ролей от формата доски и транспорта и возможностью сменить библиотеку визуализации без переписывания ядра.

«Компоновщик» как основной механизм границы с внешним форматом нецелесообразен: отделение чужого формата от ядра решается адаптером точнее.

3.9.3 Поведенческий паттерн

В смысловом ядре целесообразен паттерн «Команда». UML-диаграмма классов — на рисунке 8.

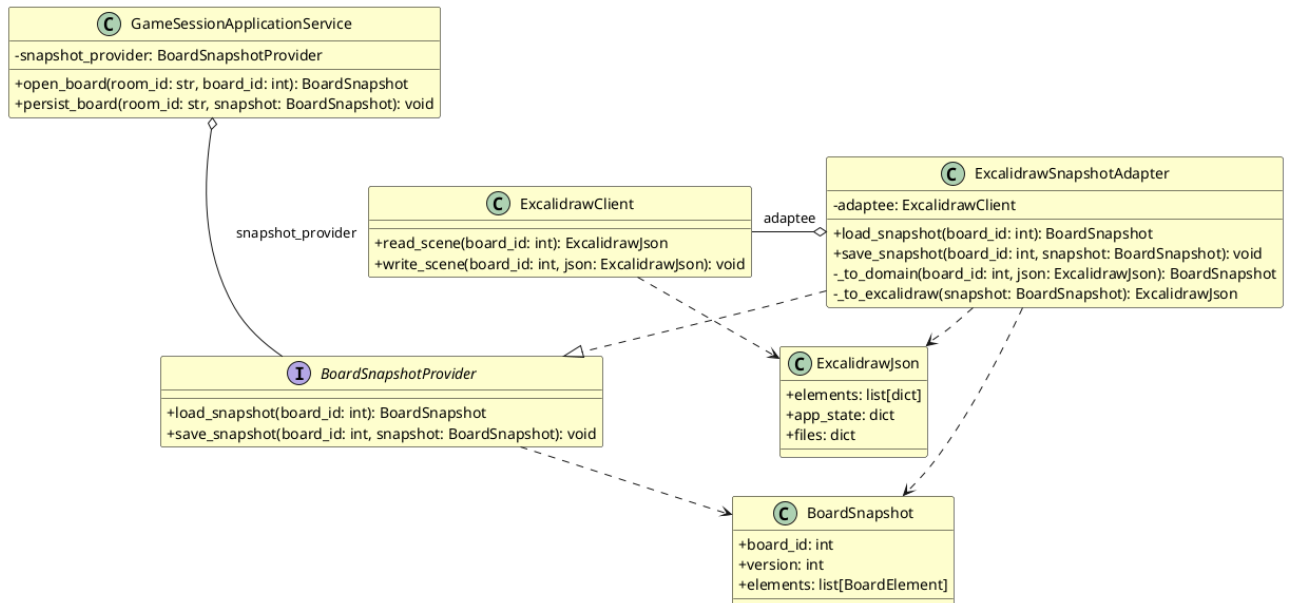


Рисунок 8 – Диаграмма классов реализации паттерна «Команда»

Допустимые действия на доске, броски и смена прав (например, только просмотр) оформляются как объекты команд с общим интерфейсом выполнения и при необходимости журналирования. Получатель — агрегат сессии комнаты; инициатор централизует порядок применения и рассылку в реальном времени. Соответствует требованиям ПрПО5 к синхронизации, истории бросков и участникам.

Выбор обоснован единообразием игровых и редакционных действий и воспроизводимостью цепочки событий.

«Посредник» как замена команде избыточен: координация уже выражена сессией комнаты и политикой прав при наличии инвокера команд.

3.10 Реализация принципов SOLID

Далее были реализованы три принципа SOLID в коде.

3.10.1 Принцип подстановки Барбар Лисков и открытости/закрытости

Чтобы соблюдался принцип подстановки Лисков (LSP), все подклассы BoardElement должны реализовывать интерфейс: согласованная семантика validate относительно RoomPolicy, предсказуемое поведение bounds, отсутствие публичных методов, доступных только у части наследников и вынуждающих в сервисах проверки типа. Различия между видами элементов выражаются данными и параметрами, а не несовместимыми наборами операций на общей ссылке BoardElement.

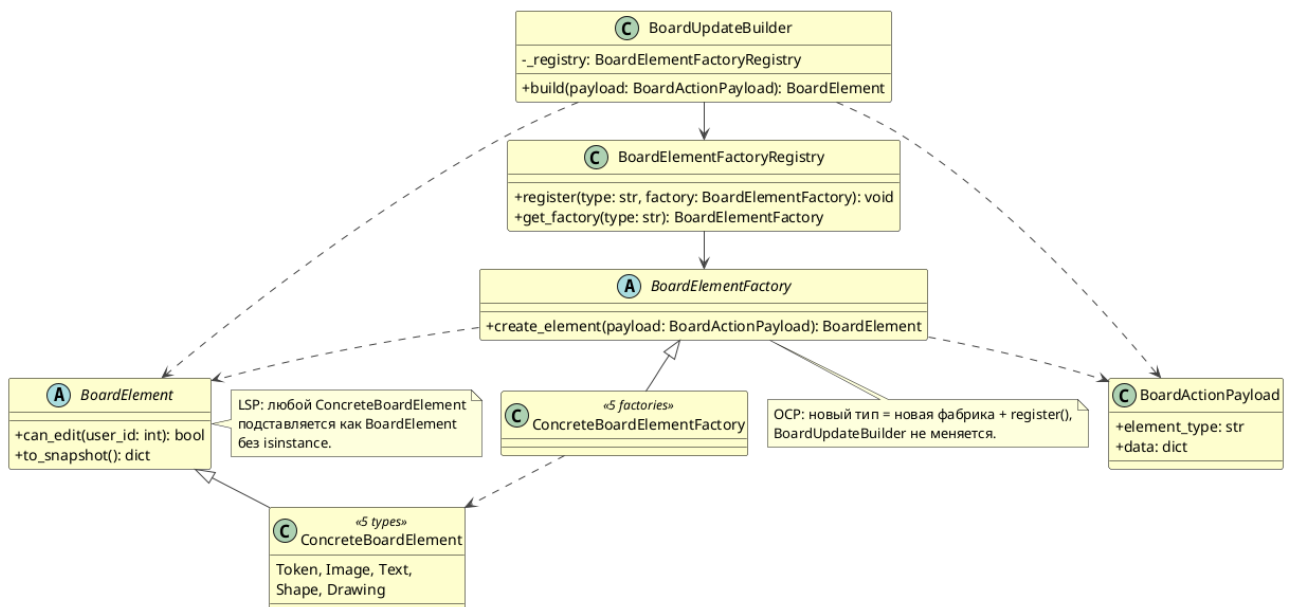


Рисунок 9 – Диаграмма кода С4 для принципа подстановки Барбары Лисков и для принципа открытости/закрытости

На диаграмме клиент построения обновления доски и реестр фабрик опираются только на абстракции BoardElement и BoardElementFactory; конкретные типы подставляются без нарушения ожидаемого поведения при синхронизации и проверке прав мастера и игрока.

3.10.2 Принцип инверсии зависимости

Прикладной уровень (исполнитель команд комнаты, сервис сохранения доски, сценарий загрузки листа в сессию) не должен зависеть от конкретной СУБД, формата файла лога или библиотеки рисования. Политика выполнения сценариев описывается абстракциями: порты аудита, снимота доски, каталога персонажей, идентичности; реализации подключаются в композиционном корне приложения. Зависимости направлены к абстракциям, заданным use case-ами (DIP).

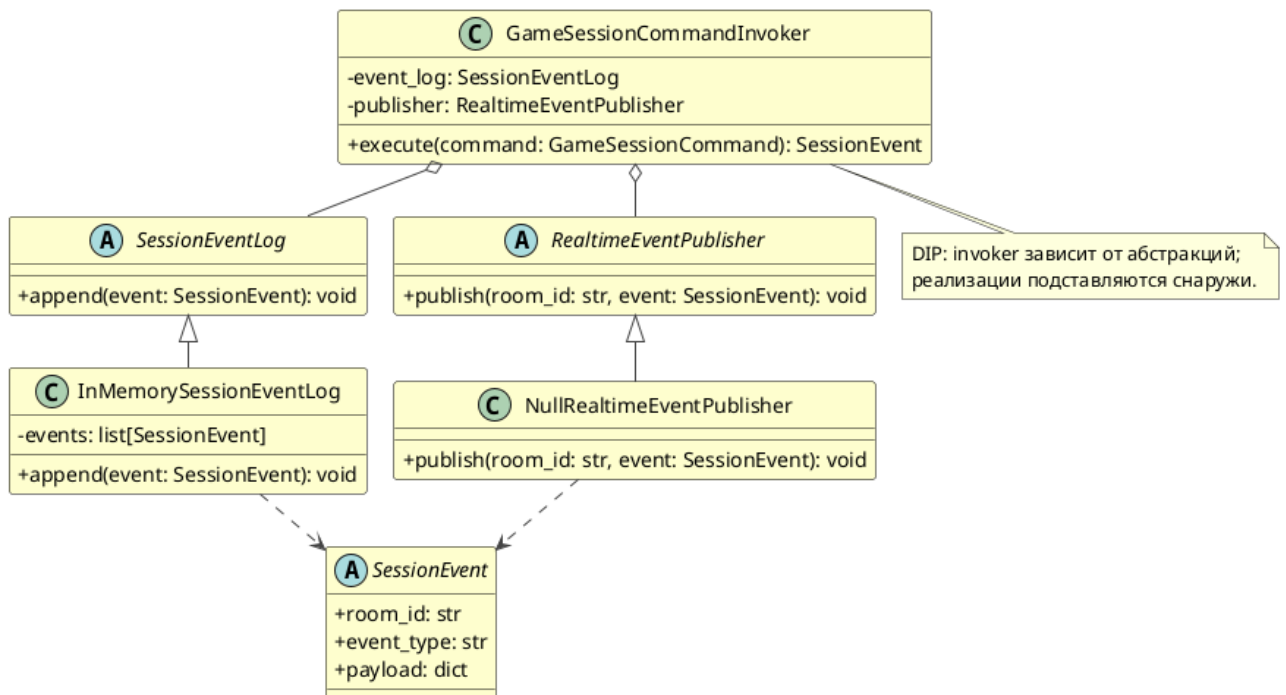


Рисунок 10 – Диаграмма кода C4 для принципа инверсии зависимостей

На диаграмме `GameSessionCommandInvoker` связан только с абстрактными `SessionEventLog` и `RealtimeEventPublisher`;

3.11 Реализация принципов организации компонентов

3.11.1 Принципы связности компонентов

Для границ выпуска и сопровождения применяется принцип согласованности повторного использования (REP): компоненты крупнее одного

класса и соответствуют зонам ответственности продукта AstRoll — учёт и доступ, персонажи и листы, игровая сессия и реалтайм-доска, сохранение снимков досок, социальные связи, журнал бросков, доставка событий в реальном времени.

На этапе эволюции функционала оправдан принцип согласованности по общей причине изменения (ССР): классы, меняющиеся по одному поводу, группируются в одном компоненте, чтобы типичная задача не расплзалась по всей системе.

3.11.2 Принципы сочетаемости компонентов

Основными остаются принцип ацикличности зависимостей (ADP) и принцип устойчивых зависимостей (SDP): граф зависимостей между компонентами без циклов; направление зависимостей — от менее устойчивых к более устойчивым (абстракции и политики комнаты и доски ближе к центру, интеграции и транспорт на периферии).

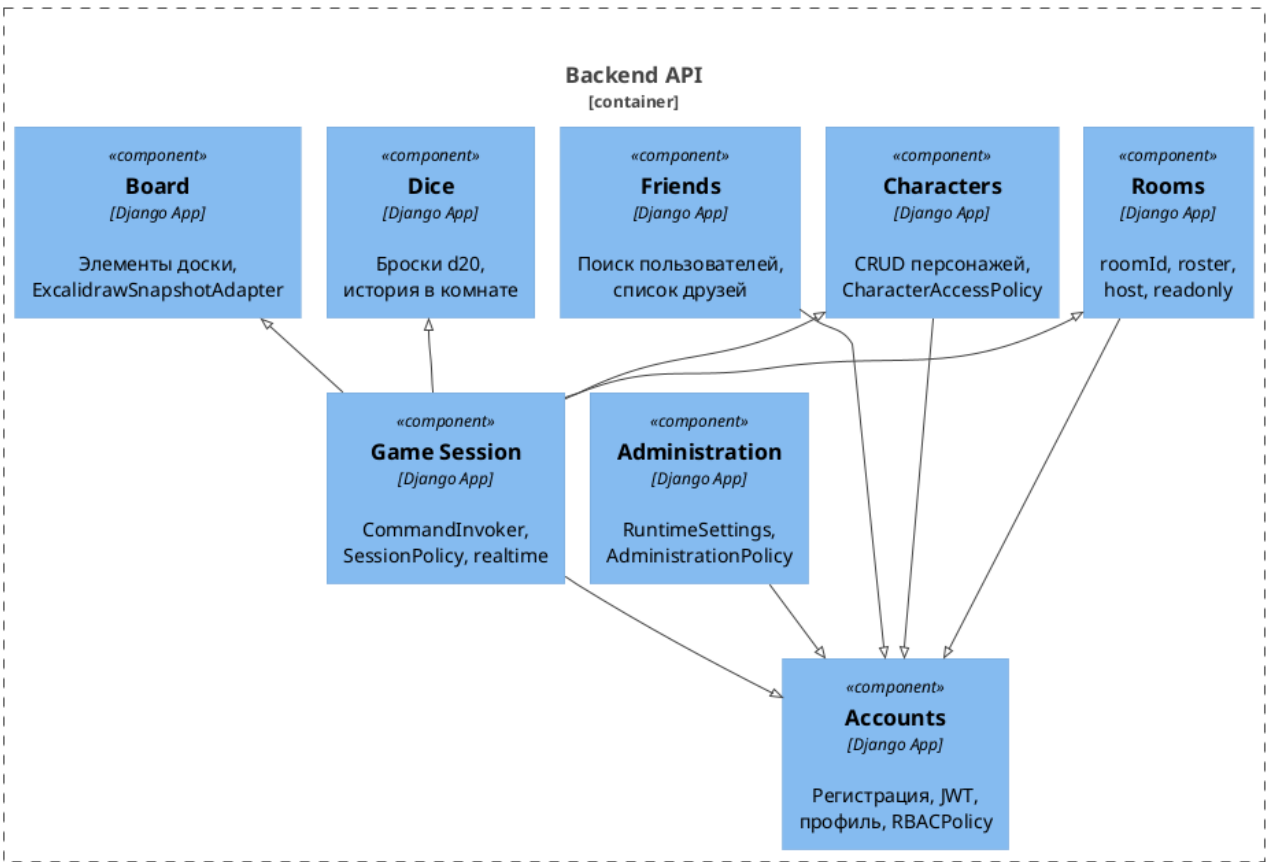


Рисунок 11 – Диаграмма компонентов С4 данной системы

Узловые зависимости задаются так: при появлении аналитики или отчётов компонент отчётов потребляет стабильные контракты ядра; игровая сессия зависит от портов хранения доски и идентичности, но не наоборот; адаптер формата доски и драйвер сокетов зависят от прикладных портов, а не вторгаются в домен с деталями транспорта. Для проверки SDP допускается оценка метрики устойчивости по осям зависимостей и контроль её невозрастания по стрелкам от периферии к ядру.

3.12 Реализация аспектов безопасности

Далее были реализованы три способа обеспечения безопасности проекта.

3.12.1 Реализация управления доступом

Первый способ — ролевая модель доступа (RBAC) и политики на уровне домена. В системе три роли: Игрок, Мастер, Администратор.

Игрок — создаёт персонажей, подключается к комнате, редактирует доску при наличии прав, делает броски;

Мастер — создаёт комнату, управляет участниками (readonly, kick), ведёт сессию;

Администратор — управляет пользователями и эксплуатационными настройками.

Для усиления безопасности политики доступа вынесены отдельно от логики хранения аккаунтов.

Компромиссы:

Дополнительная проверка прав на каждый запрос к защищённому ресурсу;

При недоступности модуля accounts часть функций, требующих проверки роли, становится недоступной.

Для смягчения можно кешировать роли в памяти на время сессии.

3.12.2 Реализация шифрования данных

Второй способ — безопасное хранение учётных данных. Пароли не хранятся в открытом виде: по NFRQ-1.1 применяется хеширование (bcrypt). JWT

подписывается секретом из переменных окружения (NFRQ-1.6). В продакшене передача данных защищается TLS на уровне API Gateway или reverse-proxy.

Компромиссы:

Хеширование и проверка пароля добавляют вычислительные затраты при входе;

Шифрование снапшотов досок и портретов в object storage (MinIO) повышает безопасность, но снижает производительность — применяется выборочно к наиболее критичным данным.

3.12.3 Реализация изоляции компонентов

Третий способ — контейнеризация и сетевое разделение. В ASTROLL/docker-compose.yml каждый сервис (auth-accounts, session, board, realtime-worker и др.) развёрнут в отдельном Docker-контейнере с минимальными привилегиями. Внешний трафик проходит через API Gateway (nginx), который выполняет маршрутизацию и rate limiting (защита от DoS). Сервисы не имеют прямого доступа к чужим базам данных.

Компромиссы:

- 1) Более сложная настройка сетей и межсервисного взаимодействия;
- 2) API Gateway становится потенциальной точкой отказа (смягчается репликацией и health-check).

3.12.4 Новые диаграммы компонентов и контейнеров нотации C4

В ходе реализации трёх способов обеспечения безопасности обновлена диаграмма компонентов C4 — рисунок 12.

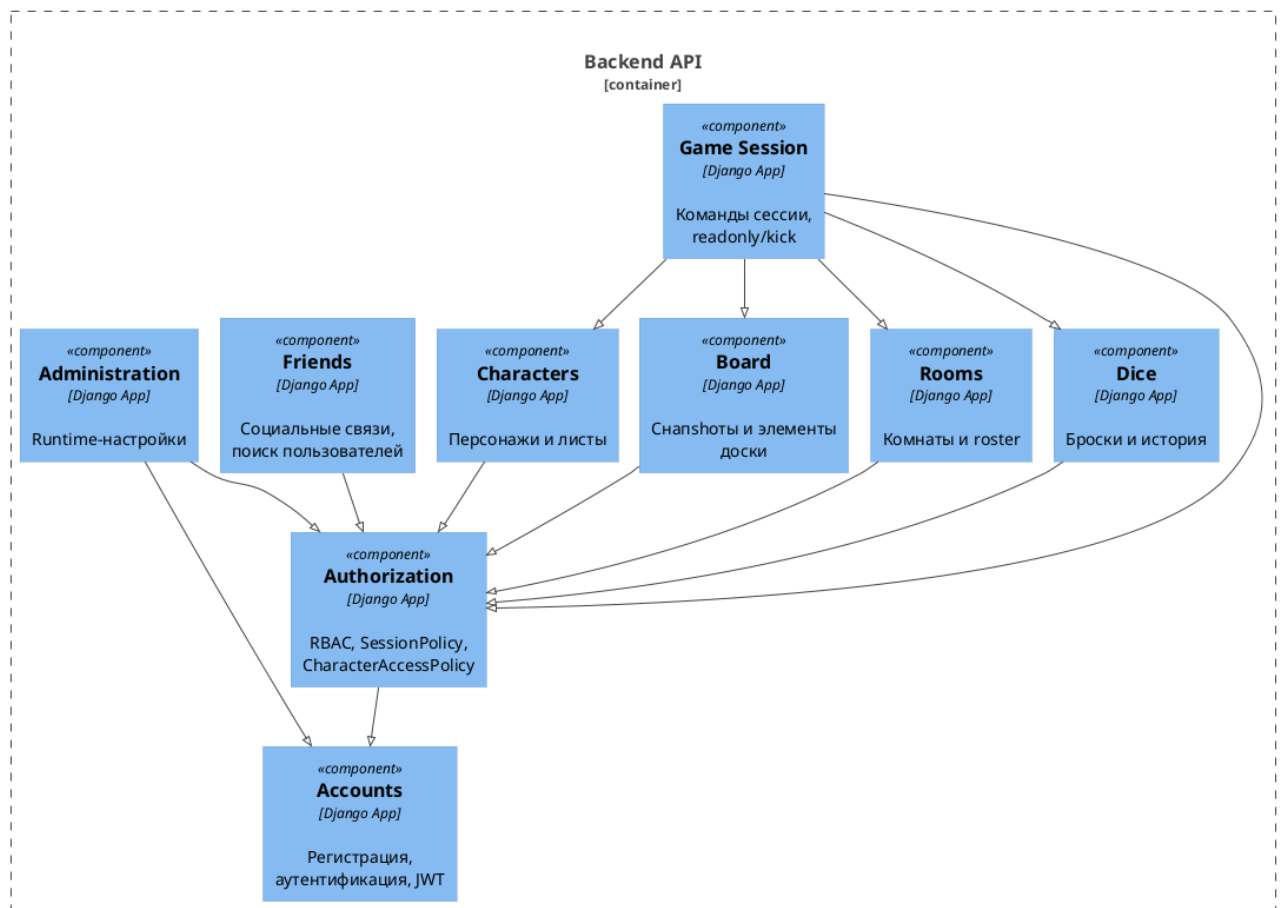


Рисунок 12 – Обновленная диаграмма компонентов нотации C4

Также составлена диаграмма контейнеров — рисунок 13.

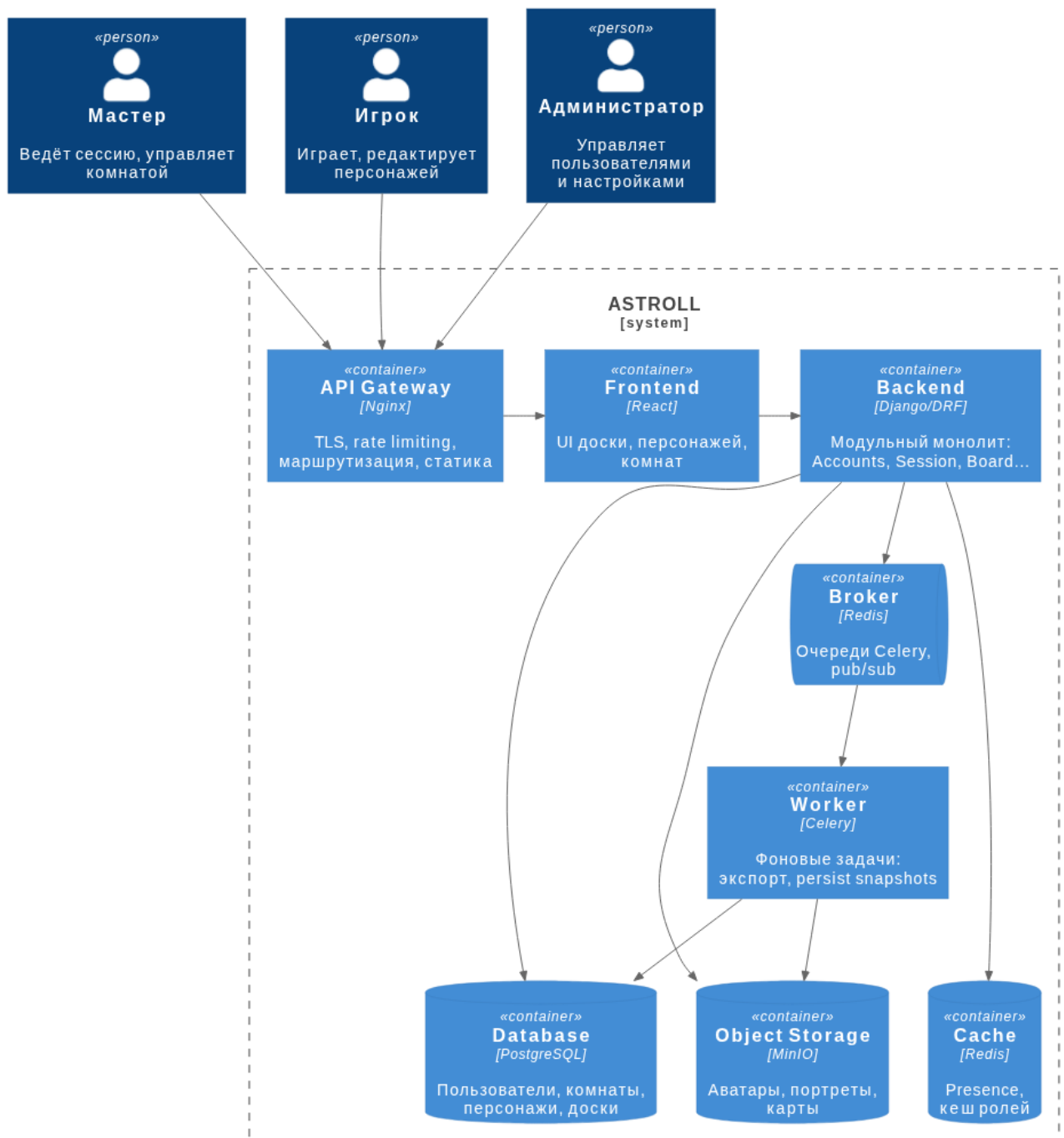


Рисунок 13 – Диаграмма контейнеров нотации C4

4 Выводы

По результатам выполнения практической работы была выбрана для проектирования система Astroll. Для визуализации взаимодействия пользователей с системой была построена UML-диаграмма вариантов использования для ролей «Мастер» и «Игрок»

Были составлены требования на уровнях:

- бизнес-требования (цели заказчика — клуба/сообщества игроков),
- требования пользователей (User Story для игрока, мастера, администратора),
- функциональные требования (аккаунт, листы, VTT, реалтайм, БД),
- нефункциональные требования (безопасность, UX, производительность, надёжность, сопровождение).

Был выбран архитектурный паттерн, был оформлен ADR, в котором был описан выбранный паттерн, был обоснован выбор и были описаны альтернативы.

Система была разобрана по методологии DDD, были выделены её поддомены (смысловое ядро, вспомогательные поддомены и поддомены общего назначения), была составлена карта контекстов по методологии DDD и диаграмма контекста нотации C4 для смыслового ядра и одного из вспомогательных поддомена.

Были реализованы паттерны проектирования каждого вида, наиболее подходящие для данной системы.

Были реализованы принципы SOLID и принципы организации компонентов каждого вида, наиболее подходящие для данной системы.

Было реализовано три способа обеспечения безопасности проекта.